Guide: 2D spatial filtering with masks up to 5×5

28 September 2026 · @Harold Molina

1. Objective and basic concepts

The student must design a procedure that traverses an image and replaces each pixel with a weighted sum of its neighbours, using a mask of up to 5×5, without using filtering functions already implemented in libraries such as numpy or other image processing libraries. This guide describes the algorithm; implementing it in Python is the student's task.

- **Image**: a matrix I with M rows × N columns. In 8-bit greyscale, each value is between 0 and 255. A colour image has 3 matrices (R, G, B channels) of the same size.
- **Mask (kernel)**: a matrix K of m × n coefficients, where m and n are odd (1, 3 or 5). It has a well-defined central pixel. Its vertical radius is a = (m − 1)/2 and its horizontal radius is b = (n − 1)/2; for a 5×5 mask, a = b = 2.
- **Neighbourhood**: the block of m × n image pixels centred on the pixel being calculated.
- **Result**: a new image O of the same size as I. Never write to I during filtering, as pixels that have already been modified would contaminate the calculation of their neighbours.

The value of each output pixel is:

$$O(x,y) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} K(i,j)\, I(x+i,\, y+j)$$

This formula is **correlation**. Strictly speaking, **convolution** uses the mask rotated by 180° (K(−i, −j)). For symmetric masks (mean, Gaussian, Laplacian), both give the same result; for asymmetric masks (Sobel, for example), the student must state which operation has been applied.

2. The mask: sizes and examples

The program must accept any mask from 1×1 to 5×5 with odd dimensions, and reject even dimensions or dimensions greater than 5 with a clear message. It is advisable to test at least one mask of each type:

Mask	Size	Coefficients	Sum	Effect
Identity	3×3	1 at the centre, 0 elsewhere	1	The output equals the input (control test)
Mean (box)	3×3 or 5×5	All 1, divided by 9 or 25	1	Smooths and blurs
Gaussian	5×5	Rows 1 4 6 4 1 / 4 16 24 16 4 / 6 24 36 24 6 / 4 16 24	1	Smooths while preserving edges better

Mask	Size	Coefficients	Sum	Effect
		16 4 / 1 4 6 4 1, divided by 256		
Laplacian	3×3	0 1 0 / 1 −4 1 / 0 1 0	0	Highlights edges; produces negative values
Horizontal Sobel	3×3	−1 0 1 / −2 0 2 / −1 0 1	0	Detects vertical edges; negative values
Sharpening (sharpen)	3×3	0 −1 0 / −1 5 −1 / 0 −1 0	1	Increases local contrast; may exceed 255

Normalisation. If the sum of the coefficients is 1, the mean brightness of the image is preserved. If it is neither 0 nor 1, the student may divide each coefficient by the sum before filtering. If the sum is 0 (edge detectors), the mask is not normalised: the result is centred on 0 and will contain negative values, which are addressed in section 5.

3. Meta-algorithm

Filtering consists of two loops over the image (rows and columns) and, for each pixel, two loops over the mask. The student must translate the following steps into Python:

1. **Load the image** using an image library and convert it into a numerical matrix. Record its dimensions M × N, the number of channels and the data type (usually unsigned 8-bit integers).
2. **Convert to a wider data type** (signed 32-bit integer or floating point) before performing any operations. With 8-bit integers, 250 + 10 gives 4 rather than 260: overflow silently "wraps around".
3. **Define and validate the mask K**: odd dimensions ≤ 5, calculation of the radii a and b, and optional normalisation (section 2).
4. **Create the output image O**, empty and of the same size as the input.
5. **For each row x (from 0 to M − 1) and each column y (from 0 to N − 1):**
    a. Set an accumulator to 0.
    b. For each offset i from −a to a and each offset j from −b to b:
        i. Calculate the neighbour's position: row x + i, column y + j.
        ii. If this position falls outside the image, **adjust it to the boundary** (section 4).
        iii. Add the product of the mask coefficient at (i, j) and the neighbour's value to the accumulator.
    c. **Adjust the accumulator to the valid range** 0–255 by saturation (section 5). If the mask sums to 0 (Laplacian, Sobel), store the sum without adjustment; rescaling is performed once traversal is complete, as it requires the minimum and maximum of the entire image.
    d. Store the result in O(x, y).
6. **Convert O to the original type** (8-bit) and display it alongside the input image for comparison; optionally, save it to disk.

Cost: M × N × m × n multiplications. For a 512 × 512 image with a 5×5 mask, this is approximately 6.5 million operations per channel, enough for the slowness of pure Python loops to become noticeable. This may be discussed as an optional improvement, but the version with explicit loops is the one that demonstrates understanding of the algorithm.

4. Boundary conditions: replicate the boundary pixel

When the mask is centred near the boundary, some of its neighbours fall outside the image. In this practical assignment, **a non-existent neighbour takes the value of the nearest boundary pixel**: the last row or column is "stretched" outwards. With a 5×5 mask, this affects the first 2 and last 2 rows and columns.

The student must apply the following rule to each neighbour coordinate separately:

- If the row is less than 0, use row 0; if it is greater than M − 1, use row M − 1.
- If the column is less than 0, use column 0; if it is greater than N − 1, use column N − 1.

Example with a 5×5 mask at the top-left corner, pixel (0, 0):

Offset	−2	−1	0	+1	+2
Requested row or column	−2	−1	0	1	2
Used row or column	0	0	0	1	2

Thus, the corner pixel I(0, 0) is counted 9 times in the window (3 repeated rows × 3 repeated columns). For an image row with values 10, 20, 30…, the row seen by the mask at the left boundary is 10, 10, **10**, 20, 30.

There are two equivalent ways to implement this, and the student chooses one:

- **Adjust indices** on the fly, within the loop, as described above.
- **Extend the image before filtering**: create a copy with a extra rows above and below and b extra columns on each side, filled by copying the boundary. Then filter only the original area, without checking indices.

The student must justify that the output is exactly the same size as the input and that no dark borders appear at the boundaries (the typical symptom of zero padding).

5. When the sum exceeds the maximum value

The weighted sum may fall outside the representable range: above 255 (sharpening or unnormalised masks) or below 0 (negative coefficients). The rule for this practical assignment is **saturation**: any value greater than 255 is set to 255 and any value less than 0 is set to 0. Before saturation, the value is **rounded** to the nearest integer.

$$O(x, y) = \min\big(255,\ \max\big(0,\ \mathrm{round}(S)\big)\big)$$

Example: with the sharpening mask, a pixel with a value of 200 surrounded by four neighbours with a value of 100 gives 5 × 200 − 4 × 100 = 600, which is stored as 255.

Points the student must take care with:

- **Apply saturation only at the end**, to the complete sum, never to each partial product: a subsequent negative term could bring the sum back within range.
- **Accumulate in a wider type** (step 2 of the algorithm). If accumulation uses 8 bits, the value wraps around rather than saturating (256 becomes 0), and black pixels appear in bright areas.
- **Masks with a sum of 0** (Laplacian, Sobel): do not apply saturation, as this would erase the negative half of the edges. Apply **linear rescaling**: first calculate and store all sums S without adjustment; then find their minimum Smin and maximum Smax across the entire image; finally, transform each pixel using O = round(255 × (S − Smin) / (Smax − Smin)). This maps the minimum to 0, the maximum to 255 and the absence of an edge to an intermediate grey. If Smax = Smin (an image with no variation), the entire output is 0.

The maximum possible sum can be anticipated: 255 × the sum of the positive coefficients. For the normalised mean or Gaussian mask, it is exactly 255, so they never saturate; for sharpening, it reaches 255 × 5 = 1275.

6. One or more masks, colour images and the student's work

Multiple masks. The program must be able to receive a list of masks and apply them in two ways:

- **In parallel**: apply each mask to the original image, displaying the results side by side for comparison.
- **In sequence**: the output of one filter is the input to the next (for example, Gaussian followed by Laplacian). Each stage applies its own boundary handling and saturation, so the order matters: the student must verify that smoothing before edge detection reduces noise.

Colour images. Filter each channel (R, G, B) separately with the same mask, applying boundary handling and saturation to each, and recombine the three. A valid alternative is to convert to greyscale first and filter a single channel.

What the student must do and submit:

- ☐ Implement the filtering algorithm for parallel execution, using the Pool class from the multiprocess library, with medium-grained parallelism and shared memory, without using library filtering or convolution functions.
    - The main function must conform to the following Python definition:

```
def image_filter( image: np.ndarray,
                  filter: np.ndarray,
                  numprocs: int,
                  is_conv: int,
                  norm: int ) -> np.ndarray
```

Where:

- *image* is an RGB image with integer values between 0 and 255,
- *filter* is the filter mask to be applied, with maximum dimensions of 5x5,
- *numprocs* is the number of computing cores that may be used to parallelise the process
- *is_conv*: has a value of 0 if the formula is applied as correlation or as convolution

        - *norm*: 1 indicates whether the output is to be normalised and 0 whether it is to be saturated.
    - It must return the filtered image as a NumPY matrix.
    - Within this function, the shared memory infrastructure must be created, the process Pool must be properly initialised with the information required for parallelisation, the filtering function must be invoked, and the filtered image must be retrieved from shared memory.
    - The student must decide whether a lock on shared memory is necessary and justify the decision in the conclusions.
- ☐ Validate the mask (odd dimensions, from 1×1 to 5×5) and provide normalisation.
- ☐ Implement boundary replication and demonstrate it using a corner pixel.
- ☐ Accumulate in a wider type, round and saturate to 0–255; for masks with a sum of 0, rescale between the minimum and maximum.
- ☐ Test the identity mask: the output must be identical to the input, pixel by pixel.
- ☐ Test an image with a constant value (for example, 128 throughout) with the 5×5 mean mask: the output must remain 128 throughout the image, including the boundaries. If the boundaries change, replication is incorrect.
- ☐ Test a very bright image (for example, 250 throughout) with the sharpening mask and verify that no dark pixels appear.
- ☐ Apply at least three different masks to a real image, in parallel and in sequence, and briefly discuss the effect of each. For parallel execution, a function with the following signature must be implemented:

```
def parallel_filtering(image: np.ndarray,
                       filter1: np.ndarray,
                       filter2: np.ndarray,
                       numprocs: int,
                       is_conv: int,
                       norm: int ) ->  ( np.ndarray , np.ndarray):
```

    - Where:
        - *image* is the image to be filtered
        - *filter1* is the first filter
        - filter2 is the second filter
        - numprocs: is the number of processors assigned to each filter being executed
        - is_conv: 1 if convolution or correlation is to be used for both filtering operations
        - norm: whether the output is to be normalised or saturated for both filtering operations.
    - The function must return a tuple containing the two resulting filtered images.
    - As nothing is self-evident: the previously designed image_filter function must be used for correct execution.
- ☐ Optional: compare the result with a library filtering function configured with the same boundary mode and measure the maximum difference.

For this, the student must use the provided notebook. A set of filters and the corresponding function signatures and calls are supplied-

The section evaluating functionality and performance must not be modified. Any additional modifications for testing must go at the end of the Notebook.

You must submit the library files required for correct execution (Notebook and Python libraries), and PERSONAL conclusions from each group member.